

Today we take a look at some exercises and experiments of FDIA on SE in Power Systems.

🔗 Exercise

Consider the matrix:

$$H = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 1 & 2 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 2 & 2 \end{pmatrix}$$

1. Is it possible to launch a strong FDIA that will increase by 2 units the value of the estimation of the first state variable, while leaving all the other state variables unchanged? If yes, provide the attack vector a that produces this effect.

We craft $c = \begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix}$ for $a = Hc$, and compute:

$$a = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 1 & 2 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 2 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

So, the answer is yes.

2. Is this attack given by $a = [1, 1, 0, 1, 1]^T$ undetectable? Why? If yes, what is the corresponding modification...

The condition for strong FDIA is $a = Hc$. Thus, can we find at least one vector c such that $a = Hc$ ($\exists c : a = Hc$)?

Let's call $c = \begin{pmatrix} a \\ b \\ c \end{pmatrix}$.

$$\begin{pmatrix} 2 & 0 & 0 \\ 1 & 1 & 2 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 2 & 2 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

$$\begin{cases} 2a = 1 \\ a + b + 2c = 1 \\ c = 0 \\ b = 1 \\ 2b + 2c = 1 \end{cases}$$

So we have $a = \frac{1}{2}$, $b = 1$ and $c = 0$.

This system of equation does not have a solution, as

$$a + b + 2c = 1 \implies \frac{1}{2} + 1 \neq 1.$$

Answer: the given a is not a strong FDIA.

3. Can an attacker launch a strong FDIA if he only has access to the first measurement? Why? In case the attack is possible, write an example.

Suppose an attacker can only attack the first meter, so $K = \{1\}$.

We need to craft a vector c such that only $a_1 \neq 0$, while $\forall i \neq 1 \ a_i = 0$.

$$\begin{pmatrix} 2 & 0 & 0 \\ 1 & 1 & 2 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 2 & 2 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} ? \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$\begin{cases} 2c_1 = a_1 \\ c_1 + c_2 + 2c_3 = 0 \\ c_3 = 0 \\ c_2 = 0 \\ 2c_2 + 2c_3 = 0 \end{cases}$$

No we can't, as $c_1 = 0$ and thus $a_1 = 0$, making a the trivial attack (no attack).

4. Can an attacker launch a strong FDIA having access to first and second measurement, and the operator directly measures the third state variable (with a PMU for example)?

$$K = \{1, 2\}, Q = \{3\}.$$

So, $\exists c : a = Hc ; a_i = 0 \forall i \notin K = \{1, 2\} ; c_3 = 0?$

$$\begin{pmatrix} 2 & 0 & 0 \\ 1 & 1 & 2 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 2 & 2 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ 0 \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$\begin{cases} 2c_1 = a_1 \\ c_1 + c_2 = a_2 \\ c_2 = 0 \\ 2c_2 = 0 \end{cases}$$

So $c_2 = 0$, and so $c_1 = a_2$, and so $a_1 = 2a_2$.

So, the attack vector could be:

$$a = \begin{pmatrix} 2 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Now, let's take a look at some simulations on Matlab.

We have to open the `FDIA` file, containing the Matlab script, and other files are simply functions.

On the left we have comments and lines of code, while on the right we have the result.

Basically data was taken from a real transmission network, the IEEE 30 buses test systems. They're realistic grids.

We run some code taking data, and then a WSL non-linear state estimation.

In the outputted data for voltage, the value of 0.9865 stands for p.u. (with p.u. being, for example, 500 *KV*). We also have the voltage angle (third column).

When working with a linearized model, one assumption we have is that V magnitude is at 1 p.u.

The vector z has dimension 46, so having 46 sensors in the grid.

With "DC" we mean we're now using a linearized model. We run the DC WLS estimation. We have a vector R capturing the covariance of every sensor, and W is the inverse of R (so the weight). We compute $\hat{x}$, $\hat{z}$ and then the residual r . We compute the length of the residual vector.

Let's simulate a fault on one sensor. We copy z with $z_f = z$, and we say that $z_f(5) = 10 \cdot z_f(5)$ (ten times the actual value).

We compute the new state estimation $\hat{x}_f$, the new measurement estimation $\hat{z}_f$, we compute the residual r_f and the norm $\|r_f\|$, which is much larger than the previous one (see the graph). *Why also other residual are spiking, even if we tamper only with the fifth sensor? Answer at home.*

By taking a look at the plot of $\hat{x}$ and $\hat{x}_f$, we can see how they're pretty similar in value. We have some impact, but not dramatic.

Thus, unstructured faults (or naive attacks) are easy to detect, as they have a big impact on the residual, and they don't create problems, as they do not have big impact on state estimation.

We also perform data correction by removing all the measurement with a residual higher than 0.06. After correcting, the new residuals are much lower.

Now, let's simulate a FDIA. We consider a perfect attack, $a = Hc$, and we build an attack vector that will increase by 5 degrees all the state variables ($\frac{\pi}{36}$ radian).

The c vector will be the vector of ones, multiplied by $\frac{\pi}{36}$. We compute thus $a = Hc$ and $z_a = z + a$. We compute $\hat{x}_a$, $\hat{z}_a$ and r_a . We expect to see two things: no impact on the residual (as the attack is a strong FDIA), and the state estimated values shifted by 5 degrees. We'll see, indeed, how $\|r_a\| = \|r\|$.

How to compute α_i (local indicator)?

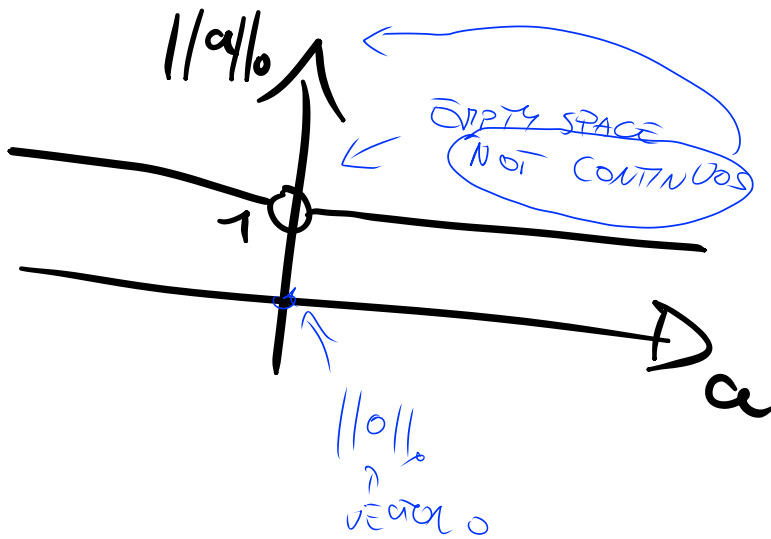
We see an example on mixed-integer (linear) programming.

WLS optimisation is an example of continuous programming (optimisation) problem. Easy to solve, because we can compute $\frac{d(\text{obj. func})}{dx} = 0$.

$$\begin{aligned} \alpha_i = & \min_c (\|a\|_0) \\ & \text{subject to} \\ & a = Hc \\ & a_i = 1 \end{aligned}$$

$\|a\|_0$ is the objective function.

Consider the simple case in which $a \in \mathbb{R}^1$. Let's plot $\|a\|_0$.



We have that c and a are vectors of real variables.

To solve this problem (through which we cannot simply derivate the function and set it to zero, as $\|a\|_0$ is not continuous) we convert the above problem into an equivalent one (having the same solution) in which boolean variables also appear. These are called mixed integer optimisation/programming problems. Some variables of the problem will be real, and others integer (in our case, boolean).

We just want to get whether the vector is zero or not.

The key idea: introduce boolean variables (one for every component of a) to indicate if the respective component a_i is zero or different from zero.

We introduce thus a new vector, a_{bin} .

$$a_{bin} = \begin{pmatrix} \delta_1 \\ \delta_2 \\ \vdots \\ \delta_m \end{pmatrix}$$

with $\delta_i = \{0, 1\} \forall i$. They're often called indicator variables, as when they're equal to one, this indicates that something particular is happening.

Suppose the condition $\delta_i = 1 \iff a_i \neq 0$ is in place, then how can we compute $\|a\|_0$? It can be written as

$$\|a\|_0 = \sum_{i=1}^m \delta_i$$

To recap: $\forall i$ we introduce boolean auxiliary variable δ_i that, when equal to one, should indicate that $a_i \neq 0$, and when equal to zero should indicate that $a_i = 0$.

To an algorithm we cannot give to a computer this sentence, so we need to translate this into a mathematical constraint that it can handle. These are called

indicator constraints, which are constraints on the indicator variables that are there to force their intended behaviour. We should enforce this:

$$\begin{aligned}\delta_i = 1 &\iff a_i \neq 0 \\ (\delta_i = 0 &\iff a_i = 0)\end{aligned}$$

We need to find mathematical formula. We use the symbols $\delta_i, a_i, \leq, M$, with M being a large positive number. This is an example of big-M constraint. We need M as δ_i can only take the values of 0 and 1, while $a_i \in \mathbb{R}$, but we can assume that $a \in [-M, M]$.

By assumption, M bound every a_i . We could get then:

$$-M\delta_i \leq a_i \leq M \cdot \delta_i$$

If $a_i \neq 0$, we have that the only value satisfying the formula is $\delta_i = 1$. If $\delta_i = 1$ instead, both $a_i = 0$ and $a_i \neq 0$ are admissible (this is a problem).

How do we fix this? Either we use more complex constraints (but we will not follow this path), or we impose $\delta_i = 1 \rightarrow a_i \neq 0$ by working on the objective function.

We introduce thus an equivalent problem for computing α_j :

$$\begin{aligned}&\min \sum_{i=1}^m \delta_i \\&\text{subject to} \\&\delta_i \in \{0, 1\} \quad i = 1, 2, \dots, m \\&a_i \leq M\delta_i \quad i = 1, 2, \dots, m \\&-a_i \leq M\delta_i \quad i = 1, 2, \dots, m \\&a_j = 1 \\&a_i \sum_{k=1}^n H_{ik} c_k \quad i = 1, 2, \dots, m \\&c_i \in \mathbb{R} \quad i = 1, 2, \dots, n\end{aligned}$$

From the second and third constraints, we get that $a_i \neq 0 \rightarrow \delta_i = 1$.

The problem is that, according to this constraint when $\delta_i = 1$, both $a_i = 0$ and $a_i \neq 0$ are feasible.

Consider the optimal solution of the above problem.

Call δ^*, a^*, c^* the optimal values for δ, a and c variables.

Can it happen that, for some i $\delta_i^* = 1$ and $a_i^* = 0$? (what we want to avoid, to make sure that $\|a\|_0 = \sum_i \delta_i$)

This can happen in a generic admissible solution a, δ, c (a, δ, c that satisfy all constraints, but in general do not minimise the objective function). But can it really happen in the optimal solution??

Missing part. Complete.

Let's see now a solving approach. Most algorithms are branch and bound algorithms.

Call P the original problem (the objective function and the constraints). We build a search tree. P contains boolean variables, so we can have a branch for $\delta_1 = 1$ and another for $\delta_1 = 0$, and we build the search tree in this way.

We thus create new nodes by fixing the value of one variable.

Suppose that, in some way, we're able to prove that the branch of $\delta_1 = 0$ will lead to higher objective function values. In this case, we can remove this branch, as we've proved that the left one is provably better.

Even choosing the order to start with makes huge difference in the algorithms.